



ZAP by
Checkmarx

ZAP by Checkmarx Scanning Report_Vulnerable

Default Policy ZAP scan on a vulnerable web app to confirm findings from balck and whitebox testing.

Site: <http://127.0.0.1:5000>

Generated on Mon, 14 Sept 2026 17:45:41

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	3
Medium	5
Low	3
Informational	5

Insights

Level	Reason	Site	Description	Statistic
Medium	Exceeded Low		Percentage of memory used	81
Low	Warning		ZAP warnings logged - see the zap. log file for details	8
Low	Exceeded High	http://127.0.0.1:5000	Percentage of slow responses	54 %
Info	Informational	http://127.0.0.1:5000	Percentage of responses with status code 2xx	39 %
Info	Informational	http://127.0.0.1:5000	Percentage of responses with status code 3xx	53 %
Info	Informational	http://127.0.0.1:5000	Percentage of responses with status code 4xx	3 %

Info	Informational	http://127.0.0.1:5000	Percentage of responses with status code 5xx	3 %
Info	Informational	http://127.0.0.1:5000	Percentage of endpoints with content type text/html	100 %
Info	Informational	http://127.0.0.1:5000	Percentage of endpoints with method GET	61 %
Info	Informational	http://127.0.0.1:5000	Percentage of endpoints with method POST	38 %
Info	Informational	http://127.0.0.1:5000	Count of total endpoints	39

Alerts

Name	Risk Level	Number of Instances
Cross Site Scripting (Persistent)	High	1
Cross Site Scripting (Reflected)	High	4
SQL Injection	High	1
Absence of Anti-CSRF Tokens	Medium	Systemic
Buffer Overflow	Medium	1
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Format String Error	Medium	1
Missing Anti-clickjacking Header	Medium	Systemic
Cookie without SameSite Attribute	Low	Systemic
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	Systemic
X-Content-Type-Options Header Missing	Low	Systemic
Authentication Request Identified	Informational	1
GET for POST	Informational	2
Session Management Response Identified	Informational	15
User Agent Fuzzer	Informational	Systemic
User Controllable HTML Element Attribute (Potential XSS)	Informational	1

Alert Detail

High	Cross Site Scripting (Persistent)
Description	Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML /JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology. When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise. There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based. Non-persistent attacks and DOM-based attacks require a user to either visit a specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash. Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of an attacker's favorite targets often include message board posts, web mail messages, and web chat software. The unsuspecting user is not required to interact with any additional site/link (e.g. an attacker site or a malicious link sent via email), just simply view the web page containing the code.
URL	http://127.0.0.1:5000/search?q=ZAP
Node Name	http://127.0.0.1:5000/search (q)
Method	GET
Attack	<script>alert(1);</script>
Evidence	
Other Info	Source URL: http://127.0.0.1:5000/notes/new
Instances	1
	Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket. Phases: Implementation; Architecture and Design Understand the context in which your data will be used and the encoding that will be expected. This is especially important when transmitting data between different components, or when generating outputs that can contain multiple encodings at the same time, such as web pages or multi-part mail messages. Study all expected communication protocols and data representations to determine the required encoding strategies.

Solution	For any data that will be output to another web page, especially any data that was received from external inputs, use the appropriate encoding on all non-alphanumeric characters. Consult the XSS Prevention Cheat Sheet for more details on the types of encoding and escaping that are needed. Phase: Architecture and Design For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server. If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated. Phase: Implementation For every web page that is generated, use and specify a character encoding such as ISO-8859-1 or UTF-8. When an encoding is not specified, the web browser may choose a different encoding by guessing which encoding is actually being used by the web page. This can cause the web browser to treat certain sequences as special, opening up the client to subtle XSS attacks. See CWE-116 for more mitigations related to encoding/escaping. To help mitigate XSS attacks against the user's session cookie, set the session cookie to be HttpOnly. In browsers that support the HttpOnly feature (such as more recent versions of Internet Explorer and Firefox), this attribute can prevent the user's session cookie from being accessible to malicious client-side scripts that use document.cookie. This is not a complete solution, since HttpOnly is not supported by all browsers. More importantly, XMLHttpRequest and other powerful browser technologies provide read access to HTTP headers, including the Set-Cookie header in which the HttpOnly flag is set. Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue." Ensure that you perform input validation at well-defined interfaces within the application. This will help protect the application even if a component is reused or moved elsewhere.
	Reference https://owasp.org/www-community/attacks/xss/ https://cwe.mitre.org/data/definitions/79.html
	CWE Id 79
	WASC Id 8
	Plugin Id 40014
High	Cross Site Scripting (Reflected)
	Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML /JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology.

Description	When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise.
	There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based.
	Non-persistent attacks and DOM-based attacks require a user to either visit a specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash.
	Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of an attacker's favorite targets often include message board posts, web mail messages, and web chat software. The unsuspecting user is not required to interact with any additional site/link (e.g. an attacker site or a malicious link sent via email), just simply view the web page containing the code.
URL	http://127.0.0.1:5000/search?q=%3C%2Fp%3E%3Cscript%3Ealert%281%29%3B%3C%2Fscript%3E%3Cp%3E
Node Name	http://127.0.0.1:5000/search (q)
Method	GET
Attack	<script>alert(1);</script>
Evidence	<script>alert(1);</script>
Other Info	
URL	http://127.0.0.1:5000/notes/8/edit
Node Name	http://127.0.0.1:5000/notes/8/edit ()(content,is_public,title)
Method	POST
Attack	<script>alert(1);</script>
Evidence	<script>alert(1);</script>
Other Info	
URL	http://127.0.0.1:5000/notes/new
Node Name	http://127.0.0.1:5000/notes/new ()(content,is_public,title)
Method	POST
Attack	<script>alert(1);</script>
Evidence	<script>alert(1);</script>
Other Info	
URL	http://127.0.0.1:5000/notes/new

Node Name	http://127.0.0.1:5000/notes/new ()(content,is_public,title)
Method	POST
Attack	<script>alert(1);</script>
Evidence	<script>alert(1);</script>
Other Info	
Instances	4
Solution	Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket. Phases: Implementation; Architecture and Design Understand the context in which your data will be used and the encoding that will be expected. This is especially important when transmitting data between different components, or when generating outputs that can contain multiple encodings at the same time, such as web pages or multi-part mail messages. Study all expected communication protocols and data representations to determine the required encoding strategies. For any data that will be output to another web page, especially any data that was received from external inputs, use the appropriate encoding on all non-alphanumeric characters. Consult the XSS Prevention Cheat Sheet for more details on the types of encoding and escaping that are needed. Phase: Architecture and Design For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server. If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated. Phase: Implementation For every web page that is generated, use and specify a character encoding such as ISO-8859-1 or UTF-8. When an encoding is not specified, the web browser may choose a different encoding by guessing which encoding is actually being used by the web page. This can cause the web browser to treat certain sequences as special, opening up the client to subtle XSS attacks. See CWE-116 for more mitigations related to encoding/escaping. To help mitigate XSS attacks against the user's session cookie, set the session cookie to be HttpOnly. In browsers that support the HttpOnly feature (such as more recent versions of Internet Explorer and Firefox), this attribute can prevent the user's session cookie from being accessible to malicious client-side scripts that use document.cookie. This is not a complete solution, since HttpOnly is not supported by all browsers. More importantly, XMLHttpRequest and other powerful browser technologies provide read access to HTTP headers, including the Set-Cookie header in which the HttpOnly flag is set. Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny

	list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue." Ensure that you perform input validation at well-defined interfaces within the application. This will help protect the application even if a component is reused or moved elsewhere.
Reference	https://owasp.org/www-community/attacks/xss/ https://cwe.mitre.org/data/definitions/79.html
CWE Id	79
WASC Id	8
Plugin Id	40012

High	SQL Injection
Description	SQL injection may be possible.
URL	http://127.0.0.1:5000/search?q=ZAP%27
Node Name	http://127.0.0.1:5000/search (q)
Method	GET
Attack	ZAP'
Evidence	HTTP/1.1 500 INTERNAL SERVER ERROR
Other Info	
Instances	1
Solution	Do not trust client side input, even if there is client side validation in place. In general, type check all data on the server side. If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?' If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries. If database Stored Procedures can be used, use them. Do <i>*not*</i> concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality! Do not create dynamic SQL queries using simple string concatenation. Escape all data received from the client. Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input. Apply the principle of least privilege by using the least privileged database user possible. In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact. Grant the minimum database access that is necessary for the application.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

CWE Id	89
WASC Id	19
Plugin Id	40018

Medium	Absence of Anti-CSRF Tokens
Description	No Anti-CSRF tokens were found in a HTML submission form. A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL /form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf. CSRF attacks are effective in a number of situations, including: * The victim has an active session on the target site. * The victim is authenticated via HTTP auth on the target site. * The victim is on the same local network as the target site. CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy.
URL	http://127.0.0.1:5000
Node Name	http://127.0.0.1:5000
Method	GET
Attack	
Evidence	<form method="post">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "password" "username"].
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login
Method	GET
Attack	
Evidence	<form method="post">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "password" "username"].
URL	http://127.0.0.1:5000/notes/6
Node Name	http://127.0.0.1:5000/notes/6
Method	GET
Attack	
Evidence	<form method="post" action="/notes/6/delete" style="display:inline">

Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: ""].
URL	http://127.0.0.1:5000/register
Node Name	http://127.0.0.1:5000/register
Method	GET
Attack	
Evidence	<form method="post">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "email" "password" "username"].
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login ()(password,username)
Method	POST
Attack	
Evidence	<form method="post">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "password" "username"].
Instances	Systemic
Solution	Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, use anti-CSRF packages such as the OWASP CSRFGuard. Phase: Implementation Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script. Phase: Architecture and Design Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330). Note that this can be bypassed using XSS. Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation. Note that this can be bypassed using XSS. Use the ESAPI Session Management control. This control includes a component for CSRF. Do not use the GET method for any request that triggers a state change. Phase: Implementation

	Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/352.html
CWE Id	352
WASC Id	9
Plugin Id	10202

Medium	Buffer Overflow
Description	Buffer overflow errors are characterized by the overwriting of memory spaces of the background
URL	http://127.0.0.1:5000/register
Node Name	http://127.0.0.1:5000/register()(email,password,username)
Method	POST
Attack	yAJxDfmJqEDhATMuyOVbccxOAiMCfgtmdUBUqKlxAElbKPjXUcwPmBjuAvycMQjWKKpakSYo
Evidence	Connection: close
Other Info	Potential Buffer Overflow. The script closed the connection and threw a 500 Internal Server Error
Instances	1
Solution	Rewrite the background program using proper return length checking. This will require a recomp
Reference	https://owasp.org/www-community/attacks/Buffer_overflow_attack
CWE Id	120
WASC Id	7
Plugin Id	30001

Medium	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://127.0.0.1:5000
Node Name	http://127.0.0.1:5000
Method	GET
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login
Method	GET
Attack	
Evidence	

Attack	n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s
Evidence	
Other Info	Potential Format String Error. The script closed the connection on a /%s.
Instances	1
Solution	Rewrite the background program using proper deletion of bad character strings. This will require a recompile of the background executable.
Reference	https://owasp.org/www-community/attacks/Format_string_attack
CWE Id	134
WASC Id	6
Plugin Id	30002

Medium	Missing Anti-clickjacking Header
Description	The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
URL	http://127.0.0.1:5000
Node Name	http://127.0.0.1:5000
Method	GET
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:5000/
Node Name	http://127.0.0.1:5000/
Method	GET
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login
Method	GET
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:5000/register
Node Name	http://127.0.0.1:5000/register
Method	GET
Attack	
Evidence	
Other	

Info	
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login ()(password,username)
Method	POST
Attack	
Evidence	
Other Info	
Instances	Systemic
Solution	Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
CWE Id	1021
WASC Id	15
Plugin Id	10020

Low	Cookie without SameSite Attribute
Description	A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login ()(password,username)
Method	POST
Attack	
Evidence	Set-Cookie: session
Other Info	
URL	http://127.0.0.1:5000/notes/3/edit
Node Name	http://127.0.0.1:5000/notes/3/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	Set-Cookie: session
Other Info	
URL	http://127.0.0.1:5000/register
Node Name	http://127.0.0.1:5000/register ()(email,password,username)
Method	POST
Attack	
Evidence	Set-Cookie: session

Other Info	
Instances	Systemic
Solution	Ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.
Reference	https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-cookie-same-site
CWE Id	1275
WASC Id	13
Plugin Id	10054

Low	Server Leaks Version Information via "Server" HTTP Response Header Field
Description	The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.
URL	http://127.0.0.1:5000
Node Name	http://127.0.0.1:5000
Method	GET
Attack	
Evidence	Werkzeug/3.0.3 Python/3.14.5
Other Info	
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login
Method	GET
Attack	
Evidence	Werkzeug/3.0.3 Python/3.14.5
Other Info	
URL	http://127.0.0.1:5000/register
Node Name	http://127.0.0.1:5000/register
Method	GET
Attack	
Evidence	Werkzeug/3.0.3 Python/3.14.5
Other Info	
URL	http://127.0.0.1:5000/robots.txt
Node Name	http://127.0.0.1:5000/robots.txt
Method	GET
Attack	
Evidence	Werkzeug/3.0.3 Python/3.14.5
Other Info	
URL	http://127.0.0.1:5000/sitemap.xml

Node Name	http://127.0.0.1:5000/sitemap.xml
Method	GET
Attack	
Evidence	Werkzeug/3.0.3 Python/3.14.5
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.
Reference	https://httpd.apache.org/docs/current/mod/core.html#servertokens https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10) https://www.troyhunt.com/shhh-dont-let-your-response-headers/
CWE Id	497
WASC Id	13
Plugin Id	10036

Low	X-Content-Type-Options Header Missing
Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	http://127.0.0.1:5000
Node Name	http://127.0.0.1:5000
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://127.0.0.1:5000/
Node Name	http://127.0.0.1:5000/
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login
Method	GET

Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://127.0.0.1:5000/register
Node Name	http://127.0.0.1:5000/register
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login ()(password,username)
Method	POST
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
Instances	Systemic
Solution	Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application /web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security_Headers
CWE Id	693
WASC Id	15
Plugin Id	10021

Informational	Authentication Request Identified
Description	The given request has been identified as an authentication request. The 'Other Info' field contains a set of key=value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to "Auto-Detect" then this rule will change the authentication to match the request identified.
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login ()(password,username)
Method	POST

Attack	
Evidence	password
Other Info	userParam=username userValue=ZAP passwordParam=password referer=http://127.0.0.1:5000/login
Instances	1
Solution	This is an informational alert rather than a vulnerability and so there is nothing to fix.
Reference	https://www.zaproxy.org/docs/desktop/addons/authentication-helper/auth-req-id/
CWE Id	
WASC Id	
Plugin Id	10111

Informational	GET for POST
Description	A request that was originally observed as a POST was also accepted as a GET. This issue does not represent a security weakness unto itself, however, it may facilitate simplification of other attacks. For example if the original POST is subject to Cross-Site Scripting (XSS), then this finding may indicate that a simplified (GET based) XSS may also be possible.
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login (password,username)
Method	GET
Attack	
Evidence	GET http://127.0.0.1:5000/login?password=ZAP&username=ZAP HTTP/1.1
Other Info	
URL	http://127.0.0.1:5000/register
Node Name	http://127.0.0.1:5000/register (email,password,username)
Method	GET
Attack	
Evidence	GET http://127.0.0.1:5000/register?email=zaproxy@example.com&password=ZAP&username=ZAP HTTP/1.1
Other Info	
Instances	2
Solution	Ensure that only POST is accepted where POST is expected.
Reference	
CWE Id	16
WASC Id	20
Plugin Id	10058

Informational	Session Management Response Identified
Description	The given response has been identified as containing a session management token. The 'Other Info' field contains a set of header tokens that can be used in the Header Based

	Session Management Method. If the request is in a context which has a Session Management Method set to "Auto-Detect" then this rule will change the session management to use the tokens identified.
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login ()(password,username)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/notes/1/edit
Node Name	http://127.0.0.1:5000/notes/1/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/notes/3/edit
Node Name	http://127.0.0.1:5000/notes/3/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/notes/4/edit
Node Name	http://127.0.0.1:5000/notes/4/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/notes/6/edit
Node Name	http://127.0.0.1:5000/notes/6/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/notes/7/edit
Node Name	http://127.0.0.1:5000/notes/7/edit ()(content,is_public,title)

Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/notes/8/edit
Node Name	http://127.0.0.1:5000/notes/8/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/register
Node Name	http://127.0.0.1:5000/register ()(email,password,username)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/login
Node Name	http://127.0.0.1:5000/login ()(password,username)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/notes/1/edit
Node Name	http://127.0.0.1:5000/notes/1/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/notes/3/edit
Node Name	http://127.0.0.1:5000/notes/3/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session

URL	http://127.0.0.1:5000/notes/4/edit
Node Name	http://127.0.0.1:5000/notes/4/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/notes/6/edit
Node Name	http://127.0.0.1:5000/notes/6/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/notes/7/edit
Node Name	http://127.0.0.1:5000/notes/7/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
URL	http://127.0.0.1:5000/notes/8/edit
Node Name	http://127.0.0.1:5000/notes/8/edit ()(content,is_public,title)
Method	POST
Attack	
Evidence	session
Other Info	cookie:session
Instances	15
Solution	This is an informational alert rather than a vulnerability and so there is nothing to fix.
Reference	https://www.zaproxy.org/docs/desktop/addons/authentication-helper/session-mgmt-id/
CWE Id	
WASC Id	
Plugin Id	10112

Informational	User Agent Fuzzer
Description	Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and the hashcode of the response body with the original response.
URL	http://127.0.0.1:5000/
Node Name	http://127.0.0.1:5000/

Method	GET
Attack	Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1)
Evidence	
Other Info	
URL	http://127.0.0.1:5000/notes/1/delete
Node Name	http://127.0.0.1:5000/notes/1/delete
Method	POST
Attack	Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1)
Evidence	
Other Info	
URL	http://127.0.0.1:5000/notes/1/edit
Node Name	http://127.0.0.1:5000/notes/1/edit ()(content,is_public,title)
Method	POST
Attack	Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1)
Evidence	
Other Info	
Instances	Systemic
Solution	
Reference	https://owasp.org/wstg
CWE Id	
WASC Id	
Plugin Id	10104

Informational	User Controllable HTML Element Attribute (Potential XSS)
Description	This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.
URL	http://127.0.0.1:5000/search?q=ZAP
Node Name	http://127.0.0.1:5000/search (q)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://127.0.0.1:5000/search?q=ZAP appears to include user input in: a(n) [input] tag [value] attribute The user input found was:

	q=ZAP The user-controlled value was: zap
Instances	1
Solution	Validate all input and sanitize output it before writing to any HTML attributes.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
CWE Id	20
WASC Id	20
Plugin Id	10031